

Q1a) Explain Alpha-Beta tree search and cut off (Pruning) procedure with example.

Alpha-Beta Pruning

Alpha-Beta pruning is an optimization technique for the **Minimax algorithm** used in game playing problems such as chess, tic-tac-toe, etc.

It reduces the number of nodes evaluated in the game tree without affecting the final result. Alpha-Beta pruning improves Minimax performance by removing branches that cannot affect final decision.

Key Terms

- **Alpha (α):** Best value that MAX player can guarantee.
- **Beta (β):** Best value that MIN player can guarantee.

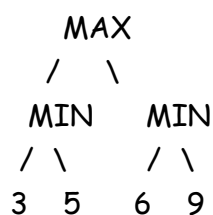
Cutoff (Pruning) Condition - When $\alpha \geq \beta$

further search is stopped because that branch cannot affect the final decision.

Working of Alpha-Beta Search

1. Start from root node.
2. MAX player tries to maximize score.
3. MIN player tries to minimize score.
4. Update α and β values while traversing.
5. If $\alpha \geq \beta$, prune remaining branches.

Example: Consider following game tree:



Step 1: Left MIN Node

- $\text{MIN}(3,5) = 3$
- Root MAX gets $\alpha = 3$

Step 2: Right MIN Node

- First child = 6
- $\beta = 6$

Now compare:

- $\alpha = 3$
- $\beta = 6$

Continue.

Second child = 9

$\text{MIN}(6,9)=6$

Root MAX chooses:

$\text{MAX}(3,6)=6$

Example of Cutoff (Pruning)



Evaluation

Left MIN:

- Left MIN - $\text{MIN}(2,4) = 2$
- $\alpha = 2$

Right MIN:

- First child = 1
- $\beta = 1$

Now: $\alpha = 2, \beta = 1$

Since: $\alpha \geq \beta$

remaining node is not evaluated (Remaining branch is pruned). This is called **cutoff or pruning**.

How Alpha-Beta Improves Performance- Reduces number of nodes evaluated, Speeds up search, Saves computation time, Allows deeper search in games

Advantages - Reduces search space, Improves speed of Minimax, Finds optimal move efficiently, Useful in AI game playing.

Applications - Chess programs, Tic-Tac-Toe, Checkers, Decision-making systems, Game Theory.

Q2b) Explain in detail the concept of backtracking and constraint propagation to solve N-Queens problem.

N-Queens Problem - The N-Queens problem is to place N queens on an $N \times N$ chessboard such that:

1. No two queens are in same row
2. No two queens are in same column
3. No two queens are on same diagonal

Backtracking Technique - Backtracking is a recursive search technique used to solve constraint satisfaction problems. It places queens one by one and checks constraints. If conflict occurs, algorithm backtracks and tries another position.

Steps of Backtracking

1. Start from first row.
2. Place queen in safe column.
3. Move to next row.
4. If no safe position exists:
 - Remove previous queen
 - Try next possible position.
5. Continue until solution found.

Algorithm

```
PlaceQueen(row):  
    if row > N:  
        print solution  
    else  
        for each column:  
            if position is safe:  
                place queen  
                PlaceQueen(row+1)  
                remove queen
```

Example: 4-Queens Problem

Step 1: Place queen at (1,1)

Q . . .
. . . .

....
....

Step 2: Try second row. Safe position $\rightarrow (2,3)$

Q...
..Q.
....
....

Step 3: Third row has no safe place. So backtrack:

- Remove queen from (2,3)
- Try another position.

This process continues until solution is found.

Final Solution

.Q..
...Q
Q...
..Q.

Advantages of Backtracking - Simple recursive approach, Guarantees solution if exists, Efficient for CSP problems

Constraint Propagation

Constraint propagation reduces search space by eliminating invalid choices early.

It uses constraints:

- Same column not allowed
- Same diagonal not allowed

Whenever queen is placed:

- Mark attacked cells unavailable.

This avoids unnecessary exploration.

Working of Constraint Propagation

Suppose queen at (1,2)

Blocked positions: Column 2, Left diagonal, Right diagonal; Only remaining safe cells are considered. Thus, Faster solution, Less computation.

Advantages of Constraint Propagation - Reduces unnecessary search, Improves efficiency, Minimizes backtracking

Applications - Chess problems, Sudoku solving, Scheduling, Resource allocation, AI constraint satisfaction problems (CSP).

Q2a) What is Constraint Satisfaction Problem (CSP)? Explain with example.

A Constraint Satisfaction Problem (CSP) is a problem in which solution is found by assigning values to variables while satisfying given constraints.

A CSP consists of:

1. **Variables**
2. **Domains** (possible values for variables)
3. **Constraints** (rules restricting values)

A CSP is represented as: $CSP = (X, D, C)$

Where, $X \rightarrow$ Set of variables, $D \rightarrow$ Domain of values, $C \rightarrow$ Set of constraints.

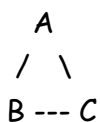
Example: Map Coloring Problem -

In map coloring: Different regions of map must be colored, Adjacent regions cannot have same color.

Colors available (Consider coloring states on a map using colors): Red, Green, Blue

Conditions: Adjacent states cannot have same color.

Example Map -



Adjacent regions: A adjacent to B and C, B adjacent to C

Formulation as CSP

Variables: $X = \{A, B, C\}$ Each variable represents a region.

Domains: $D = \{Red, Green, Blue\}$ Each region can take any color.

Constraints: $A \neq B, B \neq C, A \neq C$ Adjacent regions must have different colors.

Possible solution:

Region Color

A Red

Region Color

B Green

C Blue

All constraints are satisfied.

Steps to Solve Map Coloring CSP -

1. Select region
2. Assign valid color
3. Check constraints
4. If conflict occurs: Backtrack
5. Continue until solution found

AI Techniques Used - Backtracking, Constraint propagation, Arc consistency, Heuristic search.

Another Example: 4-Queens Problem

Place 4 queens on chessboard such that - No queens attack each other.

Constraints: No same row, No same column, No same diagonal

This is solved using: Backtracking, Constraint propagation

Characteristics of CSP -

1. Variables with finite domains
2. Constraints among variables
3. Solution satisfies all constraints
4. Used in AI search problems

Advantages - Efficient search, Structured problem representation, Reduces complexity, Useful in real-world optimization.

Applications of CSP - Sudoku solving, Timetable scheduling, Map coloring, Resource allocation, N-Queens problem, Circuit design.

Q2b) Compare and contrast stochastic games and partially observable games.

Feature	Stochastic Games	Partially Observable Games
Definition	Games with randomness or probability	Games where player cannot fully observe state
Uncertainty	Due to random events	Due to incomplete information
Information Available	Complete state visible	Partial state visible
Example	Rolling dice in Ludo	Poker, card games
Decision Basis	Probability outcomes	Belief or estimation
Environment	Fully observable	Partially observable
AI Technique Used	Probabilistic reasoning	Belief state search
Complexity	Moderate	High
Nature	Random outcomes affect game	Hidden information affects game
Real-life Example	Monopoly	Chess with hidden pieces variant
Example	Dice games, Roulette	Poker, Battleship

Q2c) How AI technique is used to solve Tic-Tac-Toe problem.

Tic-Tac-Toe Problem in AI - Tic-Tac-Toe is a two-player game played on a 3×3 grid.

Players: MAX player → X, MIN player → O

Goal: Get three symbols in row, column, or diagonal.

AI Techniques Used - State Space Search, Minimax Algorithm, Alpha-Beta Pruning, Heuristic Evaluation

Minimax Algorithm - Minimax is used to find optimal move.

- MAX player tries to maximize score.
- MIN player tries to minimize score.

Working of Minimax

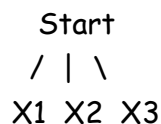
1. Generate game tree.
2. Evaluate terminal states.

3. Assign utility values:

- Win = +1
- Draw = 0
- Loss = -1

4. Choose best move.

Game Tree Example



AI evaluates all possible moves and selects best move.

Alpha-Beta Pruning - Used to reduce unnecessary node evaluation.

Benefits: Faster execution, Reduced computation

Heuristic Function - When full search is expensive, heuristic evaluates board position.

Example:

- Two X in row → positive score
- Two O in row → negative score

Steps Used by AI

1. Represent board states
2. Generate possible moves
3. Apply Minimax
4. Use Alpha-Beta pruning
5. Select optimal move

Advantages of AI in Tic-Tac-Toe - Always plays optimal move, Never loses if played correctly, Demonstrates game-search techniques

Applications - Game playing systems, Decision-making AI, Search algorithms learning, Robotics planning systems.

a) Define Game Theory. What are the components defining game as a search problem? Draw a Game Tree for Tic-Tac-Toe.

Game Theory is a branch of Artificial Intelligence and mathematics used to study decision-making in competitive situations where two or more players interact strategically.

It helps in determining the best possible move for a player considering opponent actions.

Game as a Search Problem - In AI, games are treated as search problems where:

- Each board configuration represents a state.
- Legal moves generate new states.
- AI searches for best move using algorithms like Minimax.

Components of Game as Search Problem

1. Initial State

Starting configuration of game.

Example: Empty Tic-Tac-Toe board.

2. Players

Players participating in game.

Example: MAX player (X), MIN player (O)

3. Actions / Moves

Legal moves available from current state.

Example: Place X or O in empty cell.

4. State Space

All possible game configurations.

5. Terminal Test

Checks whether game is over.

Conditions: Win, Loss, Draw

6. Utility Function

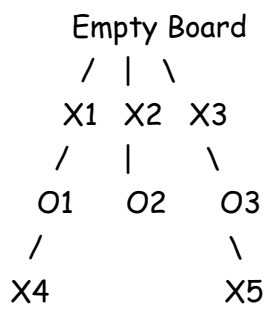
Assigns value to terminal states.

Example: Win = +1, Draw = 0, Loss = -1

7. Game Tree

Tree representing all possible moves from initial state.

Tic-Tac-Toe Game Tree



- Root node → Initial board
- Branches → Possible moves
- Leaf nodes → Final outcomes

Working of AI in Tic-Tac-Toe

1. Generate possible moves
2. Build game tree
3. Apply Minimax algorithm
4. Choose optimal move

Applications of Game Theory - Chess playing AI, Tic-Tac-Toe, Decision-making systems, Economic modelling, Robotics

Advantages - Helps in optimal decision making, Useful in competitive environments, Improves strategic planning

b) What do you understand by Constraint Propagation? Write short notes on Node Consistency and Arc Consistency.

Constraint Propagation is a technique used in Constraint Satisfaction Problems (CSP) to reduce the search space by eliminating values that violate constraints.

It propagates constraints among variables to simplify problem solving.

Purpose of Constraint Propagation

1. Reduce possible variable values
2. Detect conflicts early
3. Improve efficiency of search algorithms
4. Reduce backtracking

Example - In N-Queens problem:

- If queen is placed in one column,
- Other positions in same row, column, and diagonal are removed from consideration.

This reduces unnecessary exploration.

Types of Consistency - 1) Node Consistency, 2) Arc Consistency.

1. Node Consistency

A variable is node consistent if all values in its domain satisfy unary constraints.

Unary constraint involves only one variable.

Example -

Variable: $X = \{1,2,3,4\}$

Constraint: $X > 2$

After applying node consistency: $X = \{3,4\}$

Values 1 and 2 are removed.

Features of Node Consistency

1. Works on single variable
2. Removes invalid values
3. Simplifies domain

2. Arc Consistency

Arc consistency checks consistency between two variables connected by constraint. A variable X is arc consistent with Y if every value in X has at least one compatible value in Y .

Example -

Domains: $X = \{1,2,3\}$ $Y = \{2,3\}$

Constraint: $X < Y$

Checking:

- $X=1 \rightarrow \text{valid } (1<2)$
- $X=2 \rightarrow \text{valid } (2<3)$
- $X=3 \rightarrow \text{invalid}$

After arc consistency: $X = \{1,2\}$

Difference between Node and Arc Consistency

Node Consistency	Arc Consistency
Applied on single variable	Applied between two variables
Uses unary constraints	Uses binary constraints
Simpler	More powerful
Removes invalid domain values	Removes inconsistent pairs

Advantages of Constraint Propagation - Reduces search complexity, Improves CSP solving speed, Minimizes backtracking, Detects inconsistency early

Applications - Sudoku solving, N-Queens problem, Scheduling, Map coloring, AI planning systems.

Q2a) Explain how Minimax and Alpha-Beta algorithms change for two-player non-zero-sum games in which each player has his or her own utility function.

Non-Zero-Sum Games

In a zero-sum game: One player's gain is another player's loss.

Example: Chess, Tic-Tac-Toe

In non-zero-sum games:

- Both players may gain or lose simultaneously.
- Each player has its own utility function.

Example: Business negotiations, Resource sharing, Cooperative games

Utility Function in Non-Zero-Sum Games - Each player has separate utility values.

Example:

Outcome Player A Player B

State 1 5 3

State 2 2 6

Thus decisions are based on individual benefit.

Changes in Minimax Algorithm -

Traditional Minimax: MAX player maximizes score, MIN player minimizes same score.

Single utility function is used.

Modified Minimax for Non-Zero-Sum Games: In non-zero-sum games:

- Each player optimizes its own utility.
- No direct opposite relation exists.

Instead of one value: Each node stores utility vector.

Example: (U_A, U_B)

Where, U_A = Utility of Player A, U_B = Utility of Player B

Working

1. Generate game tree.
2. Evaluate utility for each player.
3. Each player selects move maximizing own utility.
4. Decision may involve cooperation or compromise.

Example

Start
/ \
(5,2) (3,6)

- Player A chooses left branch for utility 5.
- Player B chooses right branch for utility 6.

Changes in Alpha-Beta Pruning

Traditional Alpha-Beta

Uses: $\alpha \rightarrow$ Best value for MAX, $\beta \rightarrow$ Best value for MIN

Pruning occurs when: $\alpha \geq \beta$

Modified Alpha-Beta in Non-Zero-Sum Games

Since utilities are different:

- Single α and β values are insufficient.
- Each player maintains separate utility bounds.

Pruning becomes more complex because:

- One branch may benefit both players.
- Opponent utility is not exact opposite.

Thus, Fewer branches can be pruned, Multi-dimensional utility comparison is required.

Characteristics of Non-Zero-Sum Games - Multiple utility functions, Possibility of cooperation, Complex decision-making, More realistic than zero-sum games

Applications - Economics, Negotiation systems, Multi-agent AI, Robotics coordination, Resource management

Advantages - Models real-world situations, Supports cooperation, Better strategic planning.

Q1a) List all problem solving strategies. What is backtracking? Explain with N-Queen problem.

Problem Solving Strategies in AI - Problem solving strategies are methods used by AI to search and find solutions.

Types of Problem Solving Strategies -

1. **Generate and Test** - Generate possible solutions, Test whether solution is correct
2. **Hill Climbing** - Move toward better state step-by-step, Uses heuristic function
3. **Best First Search** - Expands most promising node first
4. **Means-End Analysis** - Reduces difference between current state and goal state
5. **Problem Reduction** - Break large problem into smaller subproblems
6. **Constraint Satisfaction** - Solve problem using constraints

Example: Sudoku, N-Queens

7. **Backtracking** - Try solution step-by-step, If failure occurs, go back and try another option
8. **Breadth First Search (BFS)** - Explores level by level
9. **Depth First Search (DFS)** - Explores deeply before backtracking

Q1b) Write Minimax Search Algorithm for two players. How use of alpha and beta cut-offs will improve performance?

Minimax Algorithm - Minimax is a decision-making algorithm used in two-player games.

It is used when:

- One player tries to maximize score (MAX)
- Other player tries to minimize score (MIN)

Examples: Chess, Tic-Tac-Toe, Checkers

Working Principle

1. Generate game tree.
2. Evaluate terminal states.
3. MAX chooses highest value.
4. MIN chooses lowest value.
5. Continue recursively.

Utility Values

Result Utility

Win +1

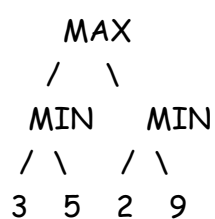
Draw 0

Loss -1

Minimax Algorithm

```
MINIMAX(state):
if terminal(state):
    return utility(state)
if MAX player's turn:
    best =  $-\infty$ 
    for each move:
        val = MINIMAX(result(move))
        best = max(best, val)
    return best
else:
    best =  $+\infty$ 
    for each move:
        val = MINIMAX(result(move))
        best = min(best, val)
    return best
```

Example Game Tree



Left MIN = 3, Right MIN = 2, MAX chooses $\max(3, 2) = 3$

Q2b) Define Constraint Satisfaction Problem (CSP). State the types of consistencies. Solve the following Crypt Arithmetic Problem.

Types of Consistencies

1. Node Consistency - A variable is node consistent if all values satisfy unary constraints.

Example: $X = \{1,2,3,4\}$

Constraint: $X > 2$

Result: $X = \{3,4\}$

2. Arc Consistency - A variable is arc consistent if every value of one variable has compatible value in another variable.

Example

Constraint: $X < Y$

Domains: $X = \{1,2,3\}, Y = \{2,3\}$

After checking: 3 in X has no valid pair.

Result: $X = \{1,2\}$

3. Path Consistency - Checks consistency among three variables.

If:

- X is consistent with Y
- Y is consistent with Z

then X should also be consistent with Z.

4. k-Consistency - A CSP is k-consistent if any consistent assignment to k-1 variables can be extended to kth variable.

5. Strong k-Consistency - A CSP is strong k-consistent if it satisfies:

- 1-consistency
- 2-consistency
- ...
- k-consistency

Solve the Crypt Arithmetic Problem -

$$\begin{array}{r} \text{B A S E} \\ + \text{B A L L} \\ \hline \text{G A M E S} \end{array}$$

Step 1: Assign Variables

Letters involved: B, A, S, E, L, G, M

Each letter represents a unique digit.

Step 2: Column-wise Addition

$$\begin{array}{r} B A S E \\ + B A L L \\ \hline G A M E S \end{array}$$

Units Column

$$E + L = S + 10c_1$$

where c_1 is carry.

Tens Column

$$S + L + c_1 = E + 10c_2$$

Hundreds Column

$$A + A + c_2 = M + 10c_3$$

$$2A + c_2 = M + 10c_3$$

Thousands Column

$$B + B + c_3 = A + 10c_4$$

$$2B + c_3 = A + 10c_4$$

Ten-thousands Column

$$G = c_4$$

Since carry can only be 1:

$$G = 1$$

One Valid Solution

$$B = 7$$

$$A = 4$$

$$S = 8$$

$$E = 3$$

$$L = 5$$

$$G = 1$$

$$M = 9$$

Verification

$$7483$$

$$+ 7455$$

$$14938$$

Thus:

$$B A S E = 7483$$

$$B A L L = 7455$$

$$G A M E S = 14938$$

Hence the solution is correct.

Q1b) Explain Monte Carlo Tree Search (MCTS) with all steps and demonstrate with one example.

Monte Carlo Tree Search (MCTS) is a heuristic search algorithm used for decision-making in Artificial Intelligence, especially in game playing.

It combines: Random sampling & Tree search to find the best possible move.

MCTS is widely used in: Chess, Go, Tic-Tac-Toe, AlphaGo systems

Main Idea of MCTS - Instead of exploring all possible states like Minimax, MCTS:

1. Randomly simulates games
2. Learns which moves are better
3. Expands promising paths more frequently

Four Steps of MCTS -

1. **Selection** - Start from root node and select the best child nodes until a leaf node is reached. Uses selection policy such as UCT (Upper Confidence Bound for Trees).
2. **Expansion** - If selected node is not terminal: Add one or more child nodes.
3. **Simulation (Rollout)** - Play random moves from expanded node until game ends.

Result may be: Win, Loss, Draw

4. **Backpropagation** - Update statistics of all visited nodes: Number of visits, Wins/losses. This information helps future decisions.

MCTS Algorithm

MCTS(root):

while time available:

 leaf = Selection(root)

 child = Expansion(leaf)

 result = Simulation(child)

 Backpropagation(child,result)

return best move

Example: Tic-Tac-Toe Suppose current board:

```
X | O | X
-----
O | X | 
-----
|  | O
```

AI has two possible moves: Move A or Move B

Step 1: Selection

Choose promising move from root. Suppose Move A selected.

Step 2: Expansion

Add child node representing Move A.

```
X | O | X
-----
O | X | X
-----
|   | O
```

Step 3: Simulation

Play random moves until game ends. Suppose AI wins.

Result: Win = +1

Step 4: Backpropagation

Update statistics: Increase visit count, Increase win score

After many simulations: Move with highest success rate is selected.

Advantages of MCTS - Efficient for large search spaces, Does not require complete game tree, Learns better moves over time, Works well in uncertain environments

Disadvantages - Computationally expensive, Random simulations may be inaccurate, Performance depends on number of simulations

Applications - Chess AI, AlphaGo, Robotics planning, Decision-making systems, Game playing agents

Monte Carlo Tree Search is a powerful AI search technique that uses Selection, Expansion, Simulation, Backpropagation to efficiently determine optimal decisions in complex games and planning problems.